

LiteGrip

LiteGrip Studio Operation Manual

NEXFORM ROBOTICS

Version: V1.0.1 Date: November 3, 2026

Revision History

Version	Date	Revision
V1.0.1	2026-11-03	Notes that factory calibration is the default; recalibration is normally unnecessary, calibrate only if it feels off
V1.0.0	2026-11-02	Created the LiteGrip Studio operation manual, generated from README.zh-CN.md in litegrip-studio

Contents

1 LiteGrip Studio 1
1.1 Related repositories 1
1.2 1. Install dependencies 1
1.3 2. Run it 2
1.3.1 Command-line options 2
1.4 3. Connect a real gripper 3
1.5 4. Calibration (usually not needed) 4
1.6 The interface 4
1.6.1 The always-present bar 5
1.6.2 Control — make the gripper move 5
1.6.3 Status — readings and faults 7
1.6.4 Plots — two charts 7
1.6.5 The rest of the window 7
1.7 Calibration 8
1.7.1 Which file the host computer uses 8
1.7.2 The gate 8
1.7.3 Automatic calibration — measure the stroke 8
1.7.4 Manual two-point calibration — record two points by hand 9
1.7.5 Choosing a file, and making it stick 9
1.7.6 What the “Source” line means 10
1.7.7 Motor zeroing — write the encoder zero 10
1.8 Troubleshooting 11
1.9 Known limitations 12
1.10 Files and settings 13
1.11 Development 14
1.12 License 14

1 LiteGrip Studio

English | 简体中文

LiteGrip Studio is the operator console for the LiteGrip two-finger adaptive gripper. Drag a slider to send the gripper to a position, press a button to open or close it fully, grasp an object at a set force limit, and watch live readings of position, force, temperature and faults. It `imports` the LiteGrip Python SDK in-process and speaks SocketCAN — there is no cross-language bridge.

With no gripper connected it runs a built-in simulated gripper, and **every control still works** — the simulation is not a UI placeholder; it runs the same motion state machine as the real hardware. So you can install it without a machine, click through every page, and get the test suite green.

The window has four tabs — Control, Status, Plots, Calibration — and the interface is in Simplified Chinese; the button names in this document are exactly the words shown on screen.

1.1 Related repositories

Repository	Role
litegrip-python	Python SDK — vendored in `src/litegrip`
litegrip-cpp	C++ SDK
litegrip-docs	Product documentation
litegrip-ros2	ROS 2 driver

1.2 1. Install dependencies

You need Python 3.10 or later, plus three libraries, installed into whichever interpreter your `python3` points at:

```
python3 -m pip install PyQt5 pyqtgraph numpy
```

For the test suite add `python3 -m pip install pytest`; for packaging add `python3 -m pip install pyinstaller`. The SDK does not need installing — it is vendored in `src/litegrip`.

Two things not to do:

- **Do not use `pip install -e .`** The version in `pyproject.toml` is a semantic-release placeholder, not a valid PEP 440 version, and setuptools will refuse to build.

- **Do not install `litegrip` from PyPI.** The fallback calibration values the host computer uses are read from the `litegrip` it imports; installing a distribution with the same name silently swaps in a different set of values.

1.3 2. Run it

```
./run_litegrip_studio.sh sim        # host computer, against the built-in simulated gripper
./run_litegrip_studio.sh gui        # host computer, against the real gripper on can0
./run_litegrip_studio.sh selftest   # self-test: no Qt, no hardware needed
./run_litegrip_studio.sh test       # test suite
./run_litegrip_studio.sh build      # single-file executable, output to dist/
```

Start with `sim`: no hardware needed, yet it runs the same code.
`selftest` runs pure logic only — unit conversion, calibration direction, blocking when the zero point belongs to another machine, plant convergence, reaching the target, torque limit, emergency-stop de-energising — and **does not import Qt at all**, so it still runs on a machine where PyQt5 is broken. It prints one `PASS` / `FAIL` line per item and a pass count at the end.
Beyond this repository you need nothing installed and no extra checkout: the SDK is vendored in `src/litegrip`, and the launch script puts `src/` on PYTHONPATH.
The launch script uses `python3`. If the dependencies are installed in a virtual environment, point `PYTHON_BIN` at it:

```
python3 -m venv .venv
.venv/bin/python3 -m pip install PyQt5 pyqtgraph numpy
PYTHON_BIN=.venv/bin/python3 ./run_litegrip_studio.sh sim
```

If you get `ModuleNotFoundError: No module named 'PyQt5'`, the dependencies went into a different interpreter. `./run_litegrip_studio.sh -h` prints the one the launch script actually uses (`current interpreter: ...`); install into that one, or point `PYTHON_BIN` at the one you installed into.
Extra arguments are passed through to the program verbatim, e.g. `./run_litegrip_studio.sh sim --log-level DEBUG`; `-h` lists all subcommands.

1.3.1 Command-line options

Option	Default	Effect
`--backend {sim,real}`	`real`	Use a simulated plant instead of the real gripper
`--can-channel NAME`	`can0`	Which SocketCAN interface to use

Option	Default	Effect
`--can-bitrate INT`	`1000000`	Configure the interface at this bitrate on connect
`--no-can-setup`	Off	Do not prepare the interface on connect, and do not show an authorisation prompt
`--can-id INT`	From the calibration file	Motor CAN ID
`--mst-id INT`	From the calibration file	Host computer CAN ID
`--calibration PATH`	The remembered path, then the bundled one	The calibration file to load
`--log-level LEVEL`	`INFO`	Console log level; the log file always records DEBUG
`--log-file PATH`	`~/.litegrip/logs/litegrip-studio.log`	Where the log is written
`--selftest`	Off	Run only the self-test that needs neither Qt nor hardware, then exit
`--version`		Print the version number and exit

1.4 3. Connect a real gripper

You do not have to bring the interface up yourself first. Press “Connect”, and the host computer looks at `can0` first, acting only when the interface really is wrong:

- Privileges are raised through `pkexec`, i.e. the desktop’s own authorisation prompt. **The host computer never sees the password at any point**, so it cannot write the password to the log either.
- If the interface is already configured, no prompt appears and no privileged command runs — so once you have brought `can0` up by hand, you are not asked again on every connection.
- The configuration includes `restart-ms 100`: a controller in bus-off restarts itself after 100 ms instead of staying down. **Not every adapter supports this.** On one that rejects it, the host computer drops the option, configures the interface anyway and says so in the log; and on such adapters a bus-off controller does not recover on its own, so the next “Connect” configures it again.
- A CAN FD interface is reported, not changed: the SDK detects FD itself and works with it, while switching the interface back to classic CAN would also affect the other nodes on that bus.

- Authorisation prompt cancelled, `can0` missing, `ip` command failed: **the connection attempt continues anyway**, and the log and banner say what happened.

If a unit file or script manages the interface and you do not want the authorisation prompt, add `--no-can-setup`; if the bitrate is not 1 Mbit, use `--can-bitrate`.
To do it by hand, the equivalent three commands are:

```
sudo ip link set can0 down
sudo ip link set can0 type can bitrate 1000000 restart-ms 100 fd off
sudo ip link set can0 up
./run_litegrip_studio.sh gui
```

If the middle one reports "Device doesn't support restart from Bus Off.", your adapter does not support `restart-ms` — drop that option and run the other two.
If the first two succeed but the last reports "RTNETLINK answers: No such file or directory", the adapter has lost its USB endpoint table, and no `ip link` command can save it. Unplug the adapter and plug it back in, or run `sudo modprobe -r gs_usb && sudo modprobe gs_usb` — the latter does the same without touching the cable, at the cost of resetting **all** `gs_usb` interfaces on the machine. When "Connect" runs into this, the host computer does it itself.

1.5 4. Calibration (usually not needed)

On startup the host computer uses the **factory calibration file bundled with the SDK**; it ships with the product and was recorded from the gripper's measured stroke. **With a real gripper attached, connected and enabled, you can use it right away; by default no recalibration is needed.**
If the gripper feels off in use — for example the position reading disagrees with the opening measured by calipers, the ends of the stroke are unreachable, motion stutters, or grasp tightness is inconsistent — then run **automatic calibration** once on the Calibration tab, replacing the factory values with data belonging to **your** gripper. Until then, treat the numbers on screen as approximate, not exact.
When the Calibration tab shows **motion gate: blocked** (BL0CKED), nothing moves until you deal with it. This is deliberate: the host computer would rather refuse motion than drive with numbers it cannot vouch for. See Calibration.

1.6 The interface

Four tabs, plus the always-visible connection bar, the emergency stop button, and the log dock at the bottom.

Tab	What is in it
Control	Position slider, action buttons, speed and grasp force
Status	Live readings, temperature, faults, link health, and motor maintenance actions
Plots	Position plots and force plots
Calibration	Calibration: two probes, file operations, gate

1.6.1 The always-present bar

Connection and power has six buttons and two status lamps:

Button	Effect
Connect	Connect to the gripper (becomes “Connecting…” while it works)
Disconnect	Disconnect
Enable	Enable the motor — nothing moves unless it is enabled
Disable	Disable the motor
Clear faults	Clear latched faults
Reset E-stop	Reset a latched emergency stop

Each button is clickable only when its command can actually be accepted, so a greyed-out “Enable” means the host computer is not connected yet, not that the button is broken.
Emergency stop (Esc) drops torque to zero and disables immediately, and it **latches**: until you press “Reset E-stop”, all motion is refused. It is bound to the **Esc** key and works even while an input field has focus or a dialog is open — it is the only shortcut deliberately allowed to interrupt anything.

1.6.2 Control — make the gripper move

Position (drag to control, release to execute) is the slider. Dragging sets the target; the command is sent only **on release**. Two markers follow it:

- **The thin cyan line** is where the gripper is heading — the last command;

- **The green triangle** is where the gripper actually is, from telemetry.

The reading beside it shows **actual position**, **target** and **error**. **While something is grasped, only the actual position is shown**: a grasp drives all the way to 0 mm at the force limit, so its target is always the closed end and its error is the object width — both true, but laid side by side they look like a motion that went wrong.

Control	Effect
Close all	One click moves to the closed position
Open all	One click moves to the maximum opening
Follow live while dragging	Send the target during the drag instead of waiting for release
Stop (hold position)	Stop and hold the current angle
Zero gravity (backdrivable)	Zero stiffness and zero torque: the motor stays enabled, but the gripper can be moved by hand

Speed is the travel speed, 5–150 mm/s, default 50 mm/s.
Grasp force is the force limit used when grasping, 3–40 N. Above 35 N it turns orange, at 40 N red; that 40 N is the mechanism's rating, not a target to push against. The lower bound is the smallest setting the console allows. Grasping goes through the SDK's torque-budgeted `grasp()`, so the old 6 N lower limit no longer applies; it is now 3 N, because on this machine 2 N and 2.5 N still barely held (2026-11-02), and the SDK refuses a grasp whose approach phase never moved when the setting is below 2.2 N. If you try to enter a smaller value, the input field stops at 3 N. Beside it are two buttons:

Button	Effect
Grasp N N	Close the gripper on the object and hold it at the set force
Release	Open to 10 mm away from the grasped object — measured from **the pose at the moment of grasping**, so those 10 mm are the clearance the object actually gets

Stop (hold position) and **Zero gravity (backdrivable)** remain available while the gate is `BLOCKED`. Close, open, grasp and release are refused, so when calibration is unavailable the only ways to move the gripper are zero gravity and your hand.

1.6.3 Status — readings and faults

Live readings shows **position**, **speed**, **grasp force**, **torque**, **speed reference**, **error**, **MOS temperature** and **coil temperature**. Temperature rows turn orange 10 °C below the limit and red at the limit (MOS 70 °C, coil 80 °C).
Link and loop shows **motion state**, **grasped**, **status frames**, **frame rate**, **link silence**, **tick duration**, **tick timeouts** and **error code**.
Connection and faults gives one link lamp and one fault lamp. When a fault is latched, the hint below says what to check: undervoltage → check the 24 V supply; overcurrent → check whether the mechanism is jammed; overtemperature → wait for the MOS or coil to cool.
Maintenance has only one action: **Zero motor…**. See Calibration.

1.6.4 Plots — two charts

Both charts share one time axis and show the last 10 seconds:

- **Position (mm)** — **measured** (solid line) against **target** (dashed). The vertical axis scales to the calibrated stroke.
- **Grasp force (N)** — **force**. The vertical axis adapts to the data in the window, so a 12 N grasp fills the whole chart instead of sitting in the bottom third of a 40 N axis.

Pause freezes them, **Clear** discards them; after resuming the buffer is empty and plotting starts from scratch, a fact the hint under the charts states.

1.6.5 The rest of the window

- Above the tabs is the **alert banner**, showing one alert at a time. Alerts that describe a state — a fault, an emergency stop, a refusal — are withdrawn automatically when the state no longer holds, so a recovered console does not keep looking broken. Alerts that report a one-off event — a failed write, a finished probe — stay until the next one displaces them. Everything, including successes, is in the log.
- At the bottom is the **log** dock. **Show debug messages** also shows debug lines in the interface; the log file records them regardless.
- **Dark** / **Light** in the status bar switches the theme. The console defaults to dark. Nothing is written until you switch, so “no one has chosen yet” and “the chosen one happens to be the default” are distinguishable.
- The status bar also shows **position** and **gate** — the gate is either `READY` or `BLOCKED`.

1.7 Calibration

1.7.1 Which file the host computer uses

There are two levels of source, and the host computer decides which one to use:

1. **The file you specify** — `--calibration`, the path remembered in the settings file, or `$LITEGRIP_CALIB`. **This is the file that describes your gripper**, and the only way off the default.
2. **The calibration file bundled with the SDK** (`litegrip/factory_calibration.json`) — when nothing is specified, this is what the host computer runs.

This repository **overrides the bundled copy with the values hand-pushed on this machine**, rather than the ones the upstream measured with a probe; the divergence is recorded in `src/litegrip/VENDORED.md`, along with its own hash.
If the specified file does not exist, or exists but is unusable, that is reported honestly and **that is the end of it** — the host computer will not quietly fall back to the factory values, because "moving on numbers from a file you did not choose, with nothing on screen to show it" is the worst of all possibilities.

1.7.2 The gate

The **motion gate** on the Calibration tab, and the **gate** in the status bar, have only two values: `READY` or `BLOCKED`. `BLOCKED` means the host computer will not send motion: there is no calibration at all, the file failed validation, the specified file does not exist, the calibration belongs to another machine, or the probe result has not been written to disk yet. The page says which. This decision is made before any frame is sent, not at the controls.
While the gate is closed, **Stop** and **Zero gravity (backdrivable)** are still available — when the file is the problem, zero gravity is how you move the gripper out of the way.

1.7.3 Automatic calibration — measure the stroke

The motor must be enabled. Press **Start automatic calibration**; the host computer drives the gripper against the open limit, then against the closed limit, asking you to confirm at each step: **reached the open limit, confirm** and **reached the closed limit, confirm**. The progress bar at the bottom of the page shows the current stage (pressing toward the open limit, backing off, pressing toward the closed limit) and the live **measured position** (in both millimetres and radians). **Cancel calibration** stops it.
The probe must press onto the hard stops — hard stops are what it is looking for — so the gripper presses once against each end of the stroke. It measures the distance between the two hard stops, and it is the only action in this program allowed to move without a calibration and to travel outside the calibrated stroke: it is the step that produces the calibration.

1.7.4 Manual two-point calibration — record two points by hand

Use this when the hard stops cannot be reached, or cannot be measured — the linkage is too tight, the drive refuses the probe step. **Start manual calibration** switches the axis to zero gravity and says so explicitly; on exit (including cancel) the host computer hands the axis back, because your hand is on the gripper the whole time. Then:

1. Push the gripper by hand to the **open** limit and press **Record open limit**.
2. Push it to the **closed** limit and press **Record closed limit**. This point is 0 mm.

Which button you press is the entire answer to "which end is zero", so buttons pressed out of order are explicitly refused rather than treated as "whichever point is due counts". Each point has a time limit of 5 minutes: the axis is loose the whole time, and a console with an "enabled but zero-stiffness" motor is a gripper that will drop onto whatever is underneath.

1.7.5 Choosing a file, and making it stick

Button	Effect
Load file…	Pick a calibration file and actually read it. The page then shows which path these numbers came from.
Refresh display	Re-read **the file currently in effect**, and only that one

The file you load is the one that loads on the next start. Loading does not expire across restarts. So when the remembered file is gone, the way out is "Load file…": pick another one, or pick the SDK-bundled `litegrip/factory_calibration.json` to fall back to factory calibration — in which case the console **forgets** the remembered path rather than writing the bundled file as "your" calibration. When the file currently in effect is deleted, "Refresh display" tells you the same thing and points at which factory calibration to load.
When the probe reaches the end it **asks you** — the dialog offers **Apply calibration** and **Not now**, and states which file the result will be written to. If you do not answer, nothing is written, because pressing "Record" asks for a **measurement**, not for motion using the measured values.

- **Apply calibration** writes to disk and has the backend read the file back; the source becomes **user calibration** and the gate turns READY. Every start from then on loads this file.
- **Not now** keeps the result in memory. The gate stays closed as before, and the gripper stays at the angle the probe found; it does not spring back.

Writing to disk refuses two things and explains why: a result that failed validation — what it would overwrite is a working calibration; and the file bundled with the SDK — it describes the gripper that recorded it. Both refusals

ask again, so “Apply calibration” is itself the retry.

1.7.6 What the “Source” line means

Label	Meaning
User calibration	A calibration file belonging to your gripper
Factory calibration	The one bundled with the SDK — default values, not a measurement of your gripper
Invalid calibration	A user file that failed validation
Uncalibrated	No file at all
In-memory calibration (unsaved)	The probe result is still in memory, not on disk; the gate stays closed until it is written

Show technical details in the same box expands the raw values the host computer is actually using.

1.7.7 Motor zeroing — write the encoder zero

In the **Maintenance** section of the **Status** tab. **Zero motor…** tells the drive that the position the gripper is at right now is the angle zero. This offset is written into the motor’s own flash, and there is **no undo** — the host computer cannot read it back, nor put it back.
It is a maintenance action, not a step of calibration: it **does not remeasure anything**. It is the right tool when the gripper and the numbers have drifted far enough apart that “measure both ends again” is the real job.
The confirmation dialog lists three consequences before writing, and the banner repeats them after:

- **Recalibration is mandatory.** The span length in the file does not change, but its two angles — and every millimetre computed from them — are now offset together. Until you redo it with automatic calibration or the manual probes, they are all wrong together.
- **The gate cannot stop it.** A zero taken at a stop shifts the entire coordinate frame by far less than the tolerance the gate checks, so motion will not be blocked. That is why the consequences are spelled out in plain words here, instead of the host computer refusing motion.
- **The motor is briefly disabled.** The SDK disables the drive, writes the offset, then enables it again, so for about a second the gripper can be pushed by hand, and Esc has no effect until the write finishes. Keep hands and workpieces clear before confirming.

Press **Write zero** to confirm. This warning is dismissed only by the **Acknowledged** button, by no other means — nothing in the loop can dismiss it, because a calibration measured with the old offset will not become correct

on its own.
Zeroing requires the host computer to be connected and the motor enabled; it is also refused while a probe is running. The simulation backend refuses outright, reporting “this backend does not support zeroing”.

1.8 Troubleshooting

What you see	What it means, what to do
`ModuleNotFoundError: No module named 'PyQt5'`	The dependencies went into a different interpreter. `run_litegrip_studio.sh -h` prints the one actually used — install into that one, or set `PYTHON_BIN`
After connecting, “Network is down, errno 100”	The interface is down or in bus-off. This is a link problem, not a motor problem: check the adapter and wiring, then reconnect
“RTNETLINK answers: No such file or directory”	The USB-CAN adapter lost its endpoint table. Unplug and replug it, or `sudo modprobe -r gs_usb && sudo modprobe gs_usb`
Pressing “Connect” does nothing and no authorisation prompt appears	The prompt was cancelled, or `can0` is managed by a unit file. Bring the interface up by hand with the three commands above
Motion gate: blocked	The host computer will not move. The page says which reason applies: no file, invalid file, missing file, or a zero belonging to another machine. “Zero gravity (backdrivable)” still works
Calibration file no longer exists: …	The remembered file was deleted. “Load file…” lets you pick another one, or the SDK-bundled `litegrip/factory_calibration.json` to fall back to factory calibration
The gripper does not move at all	Check the gate first, then whether the motor is “Enabled”, then the emergency-stop banner: a latched emergency stop refuses all motion until you press “Reset E-stop”
Motion refused, saying “no position read”	The motor has not reported any status frame yet. Enable first, then wait for **status frames** to start counting

What you see	What it means, what to do
Motor zeroing reports “this backend does not support zeroing”	You are on the simulation backend, which refuses zeroing deliberately
The force plot looks like noise	With no load, torque is only a fraction of a newton. Grasp something, or ignore it

1.9 Known limitations

- **The simulation backend refuses motor zeroing.** This is deliberate, not an omission: in the simulation that position is both the mechanism’s physical state *and* the angle it reports, so shifting only the reading would make it integrate a state that disagrees with its own account.
- **The SDK’s own `calibrate()` is not offered.** It is uninterruptible and runs about 24 seconds of motion — rather than provide a 24-second window in which the emergency stop has no effect, only the two probes on the Calibration tab are exposed.
- **The factory calibration is default values, not a measurement of your gripper.** It was recorded on the gripper it shipped with, so on another gripper of the same model the readings are approximate rather than exact, off by that unit’s manufacturing tolerances. The host computer does not try to judge this — but it does catch the other case: the file’s encoder zero belongs to a different coordinate frame (a unit that has been taken apart, or another model), because such a file’s stroke does not contain the angle the gripper actually reports, and the gate turns `BLOCKED` before any frame is sent.
- **The stroke is a constant for this machine**, `constants.DEFAULT_TRAVEL_MM`, 85.0 mm. It determines both the millimetre conversion factor and the slider range, so a console with it changed to another number reports every millimetre wrong while nothing on screen looks unusual. Moving this console to another gripper means changing this constant and re-probing — no command-line argument, setting or control can change it.
- **The force limit is the mechanical rating of 40 N**, at `constants.FORCE_MAX_N`. The theoretical 100 N appears only as a greyed-out, unselectable annotation.
- **The smallest settable force is 3 N**, at `constants.FORCE_MIN_N`. Grasping goes through the SDK’s torque-budgeted `grasp()`; on this machine 2 N and 2.5 N still barely held (2026-11-02), so the lower bound is 3 N, which also has to stay above the 2.2 N at which the SDK refuses a grasp. The lower bound is only there so that 0 still means the “no force limit” mode rather than a very light grasp. The simulator still runs the console’s own FSM, whose contact detection needs about 6 N, so a grasp near the lower bound in simulation may register contact very late; the real hardware is not subject to this.
- **The top of the range stops 1 mm inside the recorded open limit.** The probe presses the open limit out at 40 N, and the linkage yields about 1 mm under that force, so commanding straight to the recorded limit

would press the stop on every full open. The closed end gets no inset: 0 is exactly the calibrated zero.

1.10 Files and settings

The host computer keeps everything for a user under `~/.litegrip/`:

Path	What it is
`litegrip_studio.ini`	Settings: last speed and force, whether the slider follows while dragging, the remembered calibration path, theme, window geometry, the log debug switch
`litegrip_calibration.json`	The default save location for calibration — but **putting it here does not make it take effect**. The host computer reads the path it has remembered
`logs/litegrip-studio.log`	The log, rotated at 512 KiB with three kept, always recorded at DEBUG

Environment variables:

Variable	Effect
`PYTHON_BIN`	The interpreter used by the launch script and the build script
`LITEGRIP_SDK_PATH`	Use an SDK checkout instead of the vendored one; give the checkout root or its `src/`
`LITEGRIP_CALIB`	Override the default user calibration path
`LITEGRIP_FACTORY_CALIB`	Override the factory calibration file with a path you specify

---

1.11 Development

The tests need neither hardware nor an installed SDK:

```
./run_litegrip_studio.sh test
# equivalent to
QT_QPA_PLATFORM=offscreen python3 -m pytest tests/ -q
```

`QT_QPA_PLATFORM=offscreen` is why UI tests can create real widgets without a display. `tests/conftest.py` points `HOME` at a temporary directory belonging to the session, so no test can read or write your `~/.litegrip/`; the `conftest.py` at the repository root also puts `src/` into `sys.path`, which is the only step missing before the vendored SDK can be imported.

The SDK is **vendored, not a dependency**: `src/litegrip` holds a fixed commit of litegrip-python, so a clone runs with nothing installed. **Do not change any file under it** — an in-place change only survives until the next vendor, when every file is overwritten in full. If a divergence is truly unavoidable, record it file by file in `src/litegrip/VENDORED.md`; that file also holds the hash of the whole tree and the steps to re-vendor, and `tests/test_vendored_sdk.py` recomputes that hash, so a single local edit turns the tests red. Today there are no local overrides: every file there is upstream.

`./run_litegrip_studio.sh build` produces a single-file executable under `dist/`. Releases are automated by semantic-release on every merge into `main`; the git tag is the single source of truth for the version number, and `0.0.0-semantic-release` in `pyproject.toml` is a placeholder — do not edit it by hand.

All these "whys" — why the architecture looks like this, what the slider is really doing underneath, why the calibration gate sits where it does — are in `DESIGN.md` (English). This document is the user manual.

This repository follows NEXFORM ROBOTICS' shared repository standards: the agent working rules in `.nexform-standards.json`, Conventional Commits, and automated semantic-release versioning.

1.12 License

Licensed under the **Apache License 2.0** — see `LICENSE`.
Copyright © 2026 NEXFORM ROBOTICS.